The example app uses the name SIDEDKIQ_REDIS_URL to be explicit about what the purpose of the Redis instance is. Sharing a Redis with another function, like caching, is a recipe for disaster as a full cache could prevent you from queuing jobs or vice versa.

But Sidekiq doesn't know you've used this name. Rather than create an initializer to fetch the Redis URL from the environment, Sidekiq allows you to set *another* environment variable named REDIS_PROVIDER that contains the name of the environment variable that contains the Redis URL. In .env.development, set REDIS_PROVIDER to SIDEDKIQ_REDIS_URL:

snapshots/1-1/sidekiq-book/.env.test

```
DATABASE_URL=postgres://postgres:postgres@db:5432/sidekiq-book_test
SIDEKIQ_REDIS_URL=redis://redis:6379/2
» REDIS_PROVIDER=SIDEKIQ_REDIS_URL
FULLFILLMENT_API_URL=http://fake-api-server:4000/fulfillment
PAYMENTS_API_URL=http://fake-api-server:4000/payments
EMAIL_API_URL=http://fake-api-server:4000/email
ERROR_CATCHER_API_URL=http://fake-api-server:4000/error-catcher
```

You'll need to make a similar change to .env.test:

snapshots/1-1/sidekiq-book/.env.development

```
DATABASE_URL=postgres://postgres:postgres@db:5432/sidekiq-book_development
SIDEKIQ_REDIS_URL=redis://redis:6379/1
» REDIS_PROVIDER=SIDEKIQ_REDIS_URL
FULLFILLMENT_API_URL=http://fake-api-server:4000/fulfillment
PAYMENTS_API_URL=http://fake-api-server:4000/payments
EMAIL_API_URL=http://fake-api-server:4000/email
ERROR_CATCHER_API_URL=http://fake-api-server:4000/error-catcher

# Limit all requests to 5 seconds
RACK_TIMEOUT_SERVICE_TIMEOUT=5
```

This indirect mechanism may seem odd, but it is useful. It was originally designed to help developers on Heroku, who were not usually able to control the name of the environment variable for the Redis instances Heroku provided. Despite this historical curiosity, the indirection through